

Assignment 7 – notes

These notes may be updated.

7.1. Federated Learning

Traditional machine learning requires centralizing all training data in one place. What if it is not possible? (e.g. due to data privacy regulations)

This is where federated learning may help. It is a decentralized approach, where we move the training to the data instead of moving the data to the training. Standard federated learning workflow looks like this:

- The server initializes a global model and sends its starting weights to the clients.
- Each client independently trains its local copy of the model using its own private data.
 - here, we don't train the model until full convergence, but e.g. on 1 epoch of client's training data
- Once local training is complete, the clients send only their updated model weights back to the server. (The training data never leaves the client!)
- The server gathers the weight updates from all clients and aggregates them to construct a new improved global model.
 - the most basic aggregation method is federated averaging (FedAvg): we just average the weights from all clients (each client's contribution is scaled proportionally by the size of its local training dataset)
- The server sends this updated global model back to the clients, and the cycle repeats until the model reaches the desired accuracy.
 - one such cycle (sending weights to clients, local training, sending updates back to the server, and aggregation) is called a communication round

For more information see:

<https://flower.ai/docs/framework/tutorial-series-what-is-federated-learning.html>

In this task, you are going to implement federated learning for 5 clients using the Flower library (`pip install -U "flwr[simulation]"`). If Flower doesn't work on your computer, consider using Colab (with CPU runtime – of course, GPU would be better, but Colab+GPU with Flower didn't work for me).

1. Get the MNIST dataset – train and test sets (`torchvision.datasets.MNIST` with `train=True` and `train=False`). This is a classic dataset of handwritten 0-9 digits.
2. Define the architecture of neural network classifier for classifying digits (10 classes), e.g. convolutional neural network.
3. Create 5 non-IID training data splits (list of 5 `DataLoaders`), one for each client in federated learning workflow:
 - here, we imitate a real-world scenario, where clients' training datasets have different distributions
 - create these splits according to some proposed rule (make sure that they are indeed non-IID)
4. Define a Flower client class (inheriting from `fl.client.NumPyClient`) that implements:

- `__init__` constructor: ID, training `DataLoader` and local model for a given client
 - `get_parameters`: returns current local model weights as a list of NumPy arrays
 - `fit`: receives global weights, sets them on the local model, runs local training for a fixed number of epochs (e.g. 1), returns updated weights, dataset size and a dictionary containing any local metrics (this can be an empty dictionary `{}`)
5. Configure and run the Flower simulation using `fl.simulation.start_simulation` (deprecated, but useful in notebook solutions; alternatively: `fl.simulation.run_simulation`). You will need:
 - `client_fn` function: takes client ID (or `flwr.app.Context` object) as input, creates a client class instance
 - `num_clients = 5`
 - `config=fl.server.ServerConfig(num_rounds=num_rounds)` (for specified number of communication rounds)
 - `strategy`: use `fl.server.strategy.FedAvg` with the appropriate arguments
 - `evaluate_fn`: function passed to `strategy` to evaluate the global model after every round on the test set
 6. Track the federated learning model performance at each communication round.
 7. For comparison: train the classifier using standard, centralized approach – using the entire training dataset (as usual, without any federated learning).
 - use the number of epochs equal to the number of communication rounds
 - track model performance at each epoch
 8. Compare the results of federated learning and centralized learning.

7.2. Differential Privacy

Traditional machine learning algorithms can unintentionally memorize details about individual training examples. This becomes problematic when models are trained on sensitive data, since information about specific individuals may potentially be inferred from the trained model.

Differential Privacy (DP) is a mathematical framework for protecting individual privacy during data analysis and machine learning. Informally, a learning algorithm is differentially private if the inclusion or exclusion of any single training example has only a limited effect on the final model.

In deep learning, differential privacy is typically achieved by:

- computing gradients separately for individual training examples
- clipping the norm of each per-sample gradient to limit the influence of any single example
- adding calibrated random noise to the aggregated gradients before updating model parameters

The privacy guarantee is usually expressed using the privacy budget ϵ . Smaller ϵ values provide stronger privacy protection, larger ϵ values provide weaker privacy protection (case with no DP is often referred to as $\epsilon = \infty$).

In this task, you are going to apply differential privacy to neural network training for a sensitive dataset. The Opacus library is required for this task (`pip install opacus`).

1. Get the Adult Income dataset – this is the same dataset as in Problem 2.1 (SHAP). Clean the data – get rid of missing values and unnecessary columns etc. Make a standard train-test split. Prepare the data for neural network processing (scaling, encoding etc).
2. Define the architecture of neural network classifier (standard multi-layer perceptron should be enough).

- Before experimenting with DP, establish the non-DP baseline: train this classifier (as usual, without any DP) and evaluate it on test set. Track performance after each epoch. This baseline will be used in further comparison with models with DP.
- Train your classifier with DP applied, for some specified value of ϵ . Applying DP is actually very simple using Opacus:

```

1 from opacus import PrivacyEngine
2
3 ...
4
5 train_loader = DataLoader(...) #your training set DataLoader
6 model = Classifier(...) #your defined classifier
7 optimizer = optim.Adam(model.parameters(), lr=...) #or optim.SGD
8
9 privacy_engine = PrivacyEngine()
10
11 epsilon = 1 # or 5, 10, ...
12
13 delta = 1e-5 #typically less than 1/N, where N is dataset size
14 max_grad_norm = 1.0 #hyperparameter: how do we clip the gradients
15
16 model, optimizer, train_loader = privacy_engine.make_private_with_epsilon(
17     module=model,
18     optimizer=optimizer,
19     data_loader=train_loader,
20     epochs=epochs, #number of epochs for training
21     target_epsilon=epsilon,
22     target_delta=delta,
23     max_grad_norm=max_grad_norm,
24 )

```

You can now continue as usual – just use the returned `model`, `optimizer` and `train_loader` in model training. However, there is one important note.

In the code above, Opacus overwrites not only the model and optimizer, but also the training `DataLoader`. Hence, **please make sure that you re-initialize all of them (including training `DataLoader`) before each experiment** for given ϵ . Each ϵ run must be a completely independent training process.

After training your classifier with DP applied, evaluate it on test set. Track performance after each epoch.

- Run step 4 for different values of $\epsilon = 1, 5, 10$ (you can also consider more cases). After each run, do a quick sanity check using `privacy_engine.get_epsilon(delta)` to verify that the actual value of ϵ is equal or close to what you have specified.
- Compare the results for different ϵ , including the non-DP baseline from step 3. Analyze the privacy-utility trade-off. Since this is an imbalanced dataset, in addition to accuracy some other metrics may be useful (e.g. F1-score, ROC, precision-recall curves, etc.)
- This is way outside the scope of this task, but optionally you may think about simple empirical experiments that illustrate the notion of privacy, for example by trying to assess whether information about individual training samples can be inferred from the trained model and comparing the results between the non-DP and DP settings.

7.3. MLOps Pipeline

To be discussed during classes. You don't have to include the full solution for this task in your Jupyter Notebook (just very briefly describe what you have done). The presentation ("in person") of your full solution is sufficient here.